

Team Members

Name: Georgios Sidiropoulos

AM: 4789

Name: Pantelis Bonitsis

AM: 4742

Imports and Setup

```
# Basic libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Dimensionality reduction
from sklearn.decomposition import PCA
from sklearn.manifold import LocallyLinearEmbedding

# Clustering algorithms
from sklearn.cluster import KMeans, AgglomerativeClustering

# Clustering evaluation metrics
from sklearn.metrics import silhouette_score, f1_score,
confusion_matrix
from sklearn.metrics.cluster import contingency_matrix

# Preprocessing
from sklearn.preprocessing import normalize

# Deep learning
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
from tqdm.notebook import tqdm

2025-05-23 07:21:00.732614: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to
register cuFFT factory: Attempting to register factory for plugin
```

```
cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
E0000 00:00:1747984861.106559      35 cuda_dnn.cc:8310] Unable to
register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
E0000 00:00:1747984861.249102      35 cuda_blas.cc:1418] Unable to
register cuBLAS factory: Attempting to register factory for plugin
cuBLAS when one has already been registered
```

This function loads pre-extracted CNN features and labels.

Each dictionary key corresponds to a feature extractor (e.g. MobileNet or ResNet50).

```
# Feature Loading
def load_features():
    return {
        'mobilenet': {
            'features': np.load('/kaggle/input/mobilenet-features-and-
labels/mobilenet_features.npy'),
            'labels': np.load('/kaggle/input/mobilenet-features-and-
labels/labels.npy')
        },
        'resnet50': {
            'features':
np.load('/kaggle/input/resnet50-features/ResNet50.npy'),
            'labels': np.load('/kaggle/input/mobilenet-features-and-
labels/labels.npy')
        }
    }
```

Dimensionality Reduction Methods

We apply PCA, LLE, and Autoencoder to reduce the feature vectors to M dimensions ($M \in \{2, 3, 5, 10, 20\}$).

```
# Dimensionality Reduction
# This function applies dimensionality reduction using PCA, LLE and
Autoencoder for each M in M_values.
# Results are stored in a dictionary using keys like "mobilenet_PCA_2"
or "resnet50_AE_10".
def apply_reductions(features, prefix, M_values):
    reduction_methods = {}
    for M in M_values:
        # PCA
```

```

        reduction_methods[f'{prefix}_PCA_{M}'] =
PCA(n_components=M).fit_transform(features)
    # LLE
    reduction_methods[f'{prefix}_LLE_{M}'] =
LocallyLinearEmbedding(n_components=M).fit_transform(features)
    # Autoencoder
    ae, enc = build_autoencoder(features.shape[1], M)
    ae.fit(features, features, epochs=30, batch_size=64,
verbose=0)
    reduction_methods[f'{prefix}_AE_{M}'] = enc.predict(features)
    return reduction_methods

# Autoencoder Builder
# This function builds a simple symmetrical autoencoder.
# It returns both the full autoencoder model and just the encoder part
to extract compressed features.
def build_autoencoder(input_dim, encoding_dim):
    input_layer = Input(shape=(input_dim,))
    encoded = Dense(256, activation='relu')(input_layer)
    encoded = Dense(128, activation='relu')(encoded)
    bottleneck = Dense(encoding_dim, activation='linear')(encoded)
    decoded = Dense(128, activation='relu')(bottleneck)
    decoded = Dense(256, activation='relu')(decoded)
    output_layer = Dense(input_dim, activation='linear')(decoded)
    autoencoder = Model(inputs=input_layer, outputs=output_layer)
    encoder = Model(inputs=input_layer, outputs=bottleneck)
    autoencoder.compile(optimizer='adam', loss='mse')
    return autoencoder, encoder

```

2D and 3D Visualizations

We visualize reduced features in 2D or 3D space (for $M = 2$ and 3), using colors to indicate actual labels.

```

# Visualization
def plot_reduction_subplots(reduction_methods, labels):
    plot_items = [(key, val) for key, val in reduction_methods.items()]
    if val.shape[1] in [2, 3]:
        n_cols = 2
        n_rows = (len(plot_items) + 1) // n_cols
        fig = plt.figure(figsize=(12, 5 * n_rows))
        for idx, (key, val) in enumerate(plot_items):
            ax = fig.add_subplot(n_rows, n_cols, idx + 1, projection='3d')
            if val.shape[1] == 3 else None)
                if val.shape[1] == 2:
                    ax.scatter(val[:, 0], val[:, 1], c=labels,
cmap='coolwarm', s=10)

```

```

        else:
            ax.scatter(val[:, 0], val[:, 1], val[:, 2], c=labels,
                        cmap='coolwarm', s=10)
            ax.set_title(f'{key}')
            plt.tight_layout()
            plt.show()

```

Clustering and Evaluation

We Apply clustering (KMeans and Agglomerative) on reduced data and evaluate using:

- Purity
- F1-score
- Silhouette Score

```

# Computes purity between predicted clusters and true labels
def purity_score(y_true, y_pred):
    contingency = contingency_matrix(y_true, y_pred)
    return np.sum(np.amax(contingency, axis=0)) / np.sum(contingency)

# Evaluates clustering output with purity, F1 and silhouette
def evaluate_clustering(X, y_true, clustering_model):
    y_pred = clustering_model.fit_predict(X)
    purity = purity_score(y_true, y_pred)
    fscore = f1_score(y_true, y_pred, average='macro')
    sil = silhouette_score(X, y_pred)
    return purity, fscore, sil

```

Clustering Loop

We run clustering for each reduced dataset using 3 methods and for K values from 2 to 10.

```

# Iterates over all reduced representations, runs clustering, and collects evaluation results.
def run_clustering(reduction_methods, labels, prefix):
    results = []
    for name, data_reduced in tqdm(reduction_methods.items(),
                                    desc=f"Clustering on {prefix}"):
        for k in range(2, 11):
            for method in ['kmeans_euclidean', 'kmeans_cosine',
                           'agglomerative']:
                data_copy = data_reduced.copy()

```

```

        if method == 'kmeans_euclidean':
            model = KMeans(n_clusters=k, random_state=42,
n_init=10)
        elif method == 'kmeans_cosine':
            model = KMeans(n_clusters=k, random_state=42,
n_init=10)
            data_copy = normalize(data_copy)
        else:
            model = AgglomerativeClustering(n_clusters=k)
        purity, fscore, sil = evaluate_clustering(data_copy,
labels, model)
        results.append({
            'features': prefix,
            'method': name,
            'cluster_model': method,
            'k': k,
            'purity': purity,
            'fscore': fscore,
            'silhouette': sil
        })
    return results

```

Plotting Clustering Evaluation Metrics

We visualize Silhouette Score, Purity, and F1 across different K values to identify the best clustering.

```

# This function generates line plots (metric vs K) and boxplots for a
selected feature source.
def plot_metrics(results_df, source_name):
    metrics = ['silhouette', 'purity', 'fscore']

    # Filter only for the selected source
    results_df = results_df[results_df['features'] == source_name]

    for name in results_df['method'].unique():
        subset = results_df[results_df['method'] == name]
        fig, axes = plt.subplots(1, 3, figsize=(18, 4))
        for i, metric in enumerate(metrics):
            ax = axes[i]
            sns.lineplot(data=subset, x='k', y=metric,
hue='cluster_model', ax=ax)
            ax.set_title(f'{metric.capitalize()} vs K')
            ax.set_xlabel('Number of Clusters (K)')
            ax.set_ylabel(metric.capitalize())
            ax.grid(True)
        fig.suptitle(f'{name}', fontsize=14)

```

```

plt.tight_layout()
plt.show()

# Boxplots for the selected feature source
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for i, metric in enumerate(metrics):
    sns.boxplot(data=results_df, x='cluster_model', y=metric,
ax=axes[i])
    axes[i].set_title(f'{metric.capitalize()} Distribution by
Clustering Method')
    axes[i].set_xlabel('Clustering Method')
    axes[i].set_ylabel(metric.capitalize())
    axes[i].grid(True)
plt.suptitle(f'{source_name} - Metric Distributions', fontsize=16)
plt.tight_layout()
plt.show()

# Prints the best clustering configurations across all sources by
silhouette score.
def print_best_k(results_df, top=10):
    # Get top configurations overall sorted by silhouette score
    best_k = results_df.sort_values(by='silhouette',
ascending=False).head(top).reset_index(drop=True)

    print(f"\nTop {top} overall configurations by Silhouette score:")
    display_cols = ['features', 'method', 'cluster_model', 'k',
'silhouette', 'purity', 'fscore']
    print(best_k[display_cols].to_string(index=False))

```

Run for Single Feature Source

This function applies dimensionality reduction, visualizes reduced features, and evaluates clustering for a given feature extractor (e.g. MobileNet).

```

def run_for_feature_source(source_name, data, M_values):
    features = data['features']
    labels = data['labels']

    # We apply dimensionality reduction (PCA, LLE, AE)
    reduction_methods = apply_reductions(features, source_name,
M_values)

    # We plot 2D/3D visualizations for the current source
    plot_reduction_subplots(reduction_methods, labels)

    # We run clustering (KMeans, Agglomerative) and evaluate
    return run_clustering(reduction_methods, labels, source_name)

```

Setup and Execution for a Feature Extractor

```
features_dict = load_features()
M_values = [2, 3, 5, 10, 20]
all_results = []
```

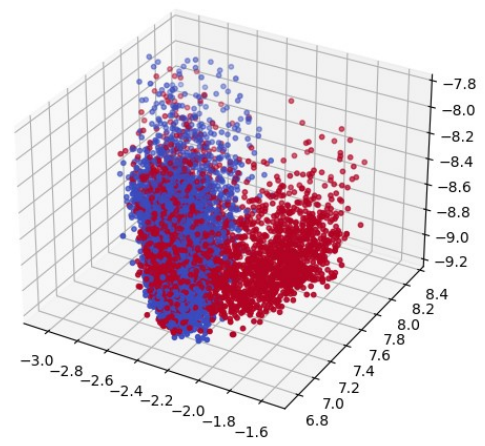
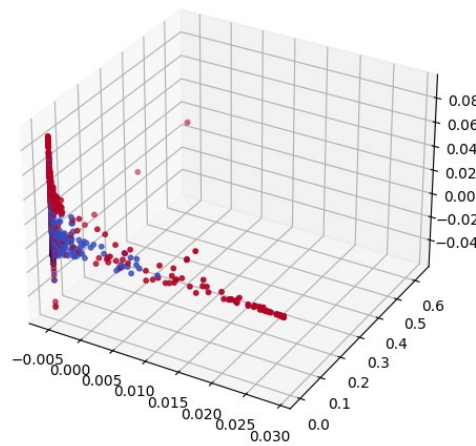
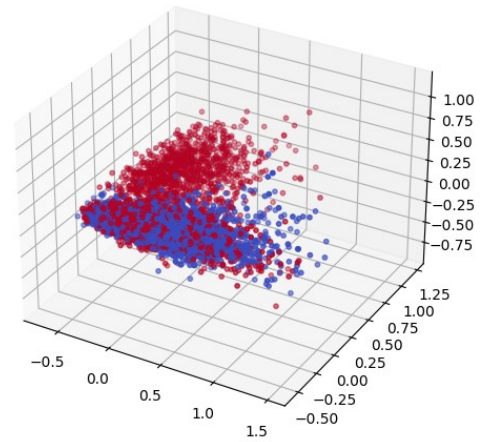
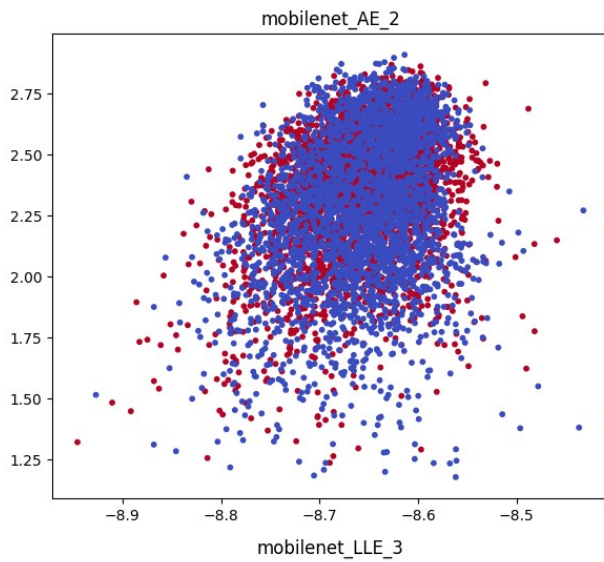
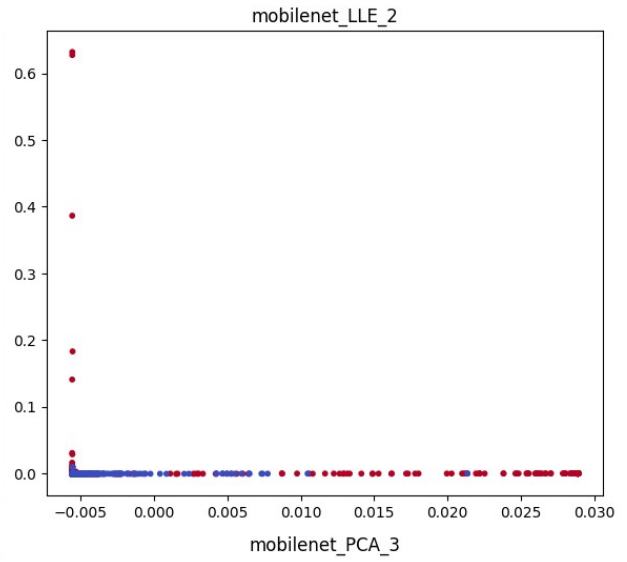
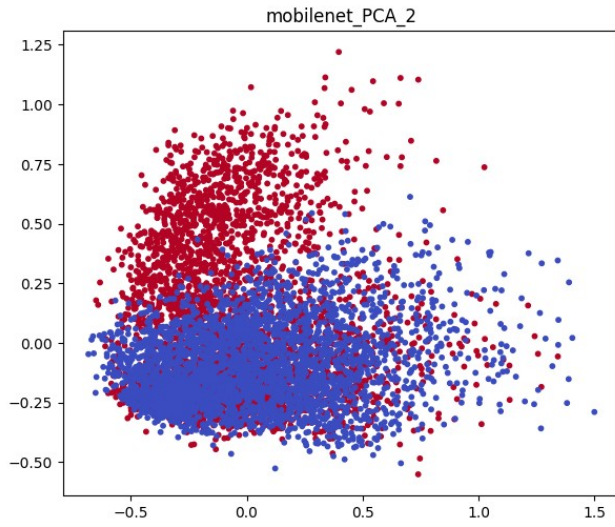
MobileNet

Run for MobileNet

```
results_mobilenet = run_for_feature_source("mobilenet",
features_dict["mobilenet"], M_values)
```

```
2025-05-23 07:21:27.721205: E
external/local_xla/xla/stream_executor/cuda/cuda_driver.cc:152] failed
call to cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN
ERROR (303)
```

```
200/200 ██████████ 0s 2ms/step
200/200 ██████████ 1s 2ms/step
200/200 ██████████ 0s 2ms/step
200/200 ██████████ 0s 2ms/step
200/200 ██████████ 0s 2ms/step
```



```
{"model_id":"a77d1117fe7c4b1fadcl4482e27030de","version_major":2,"version_minor":0}
```


Observations from MobileNet Visualizations

Based on a visual inspection of the reduced feature plots from MobileNet:

- **PCA:** Shows a clear difference between the two classes, especially in 3D.
 - **LLE:** Works better in 3D than 2D. The class separation is okay but not as strong as PCA.
 - **Autoencoder:** Does not work well in 2D. The two classes are mixed together. In 3D, it performs better and the clusters are easier to see.
-

Clustering Evaluation Plots (MobileNet)

The following plots show how each dimensionality reduction method (PCA, LLE, AE) performs on MobileNet features, across different cluster counts ($K = 2$ to 10).

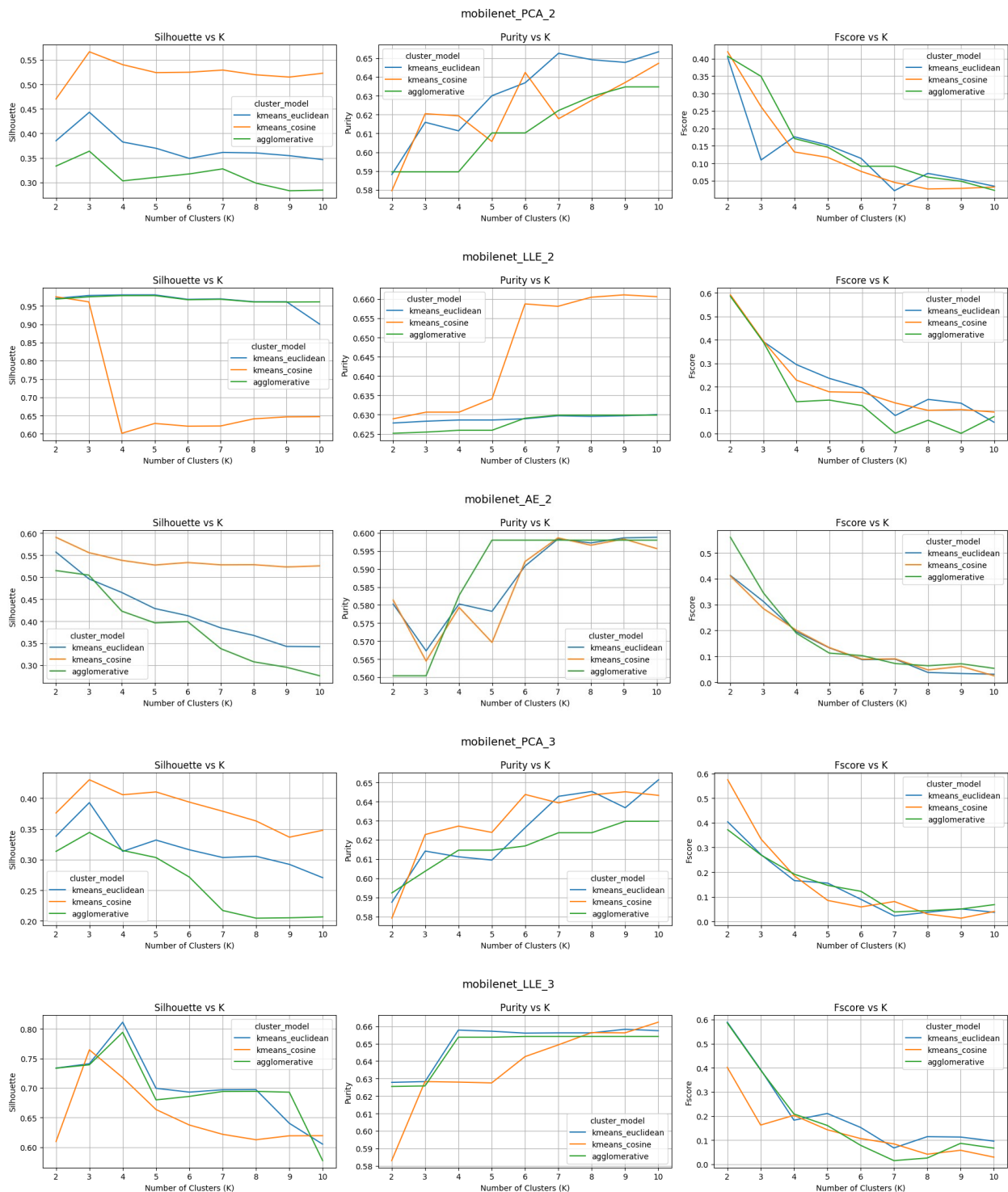
For each method and dimensionality, we display:

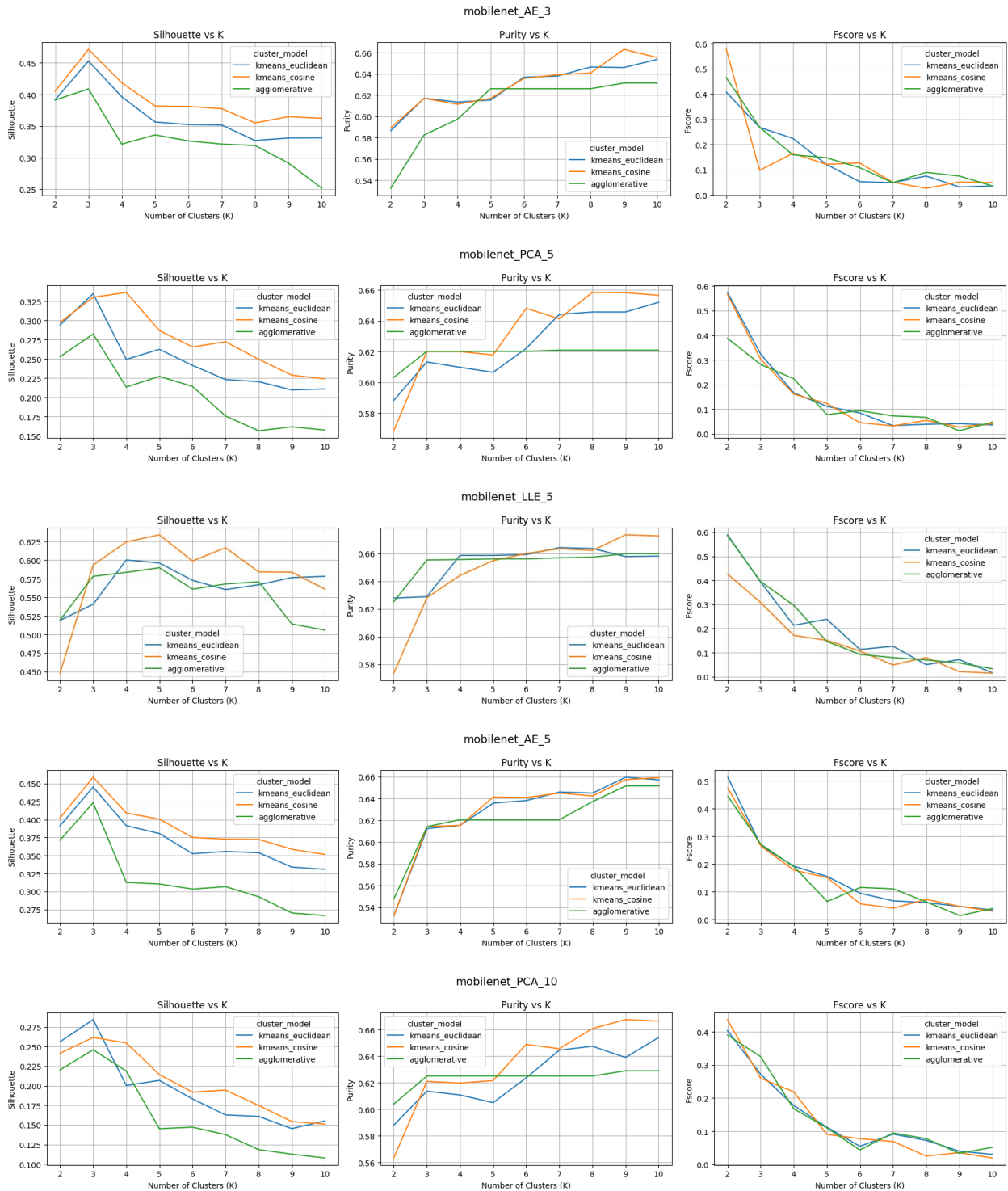
- **Silhouette Score:** how well-defined the clusters are
- **Purity:** how well the clusters match the true labels
- **F1-Score:** overall clustering accuracy

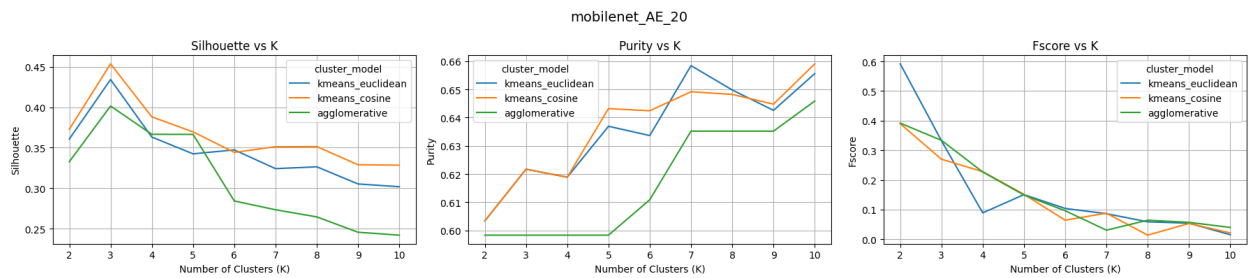
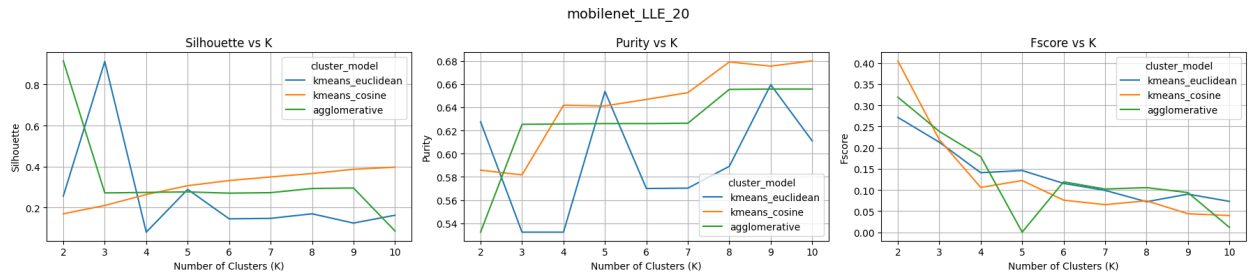
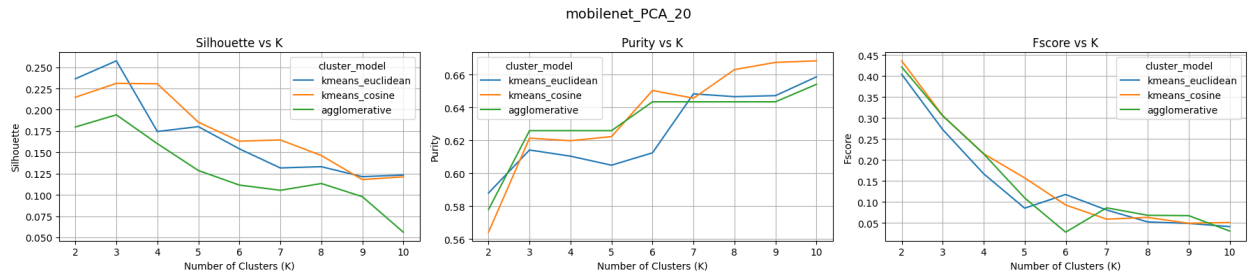
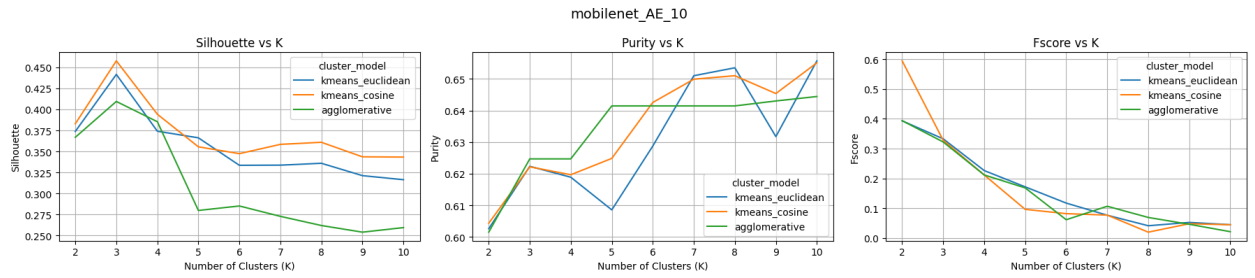
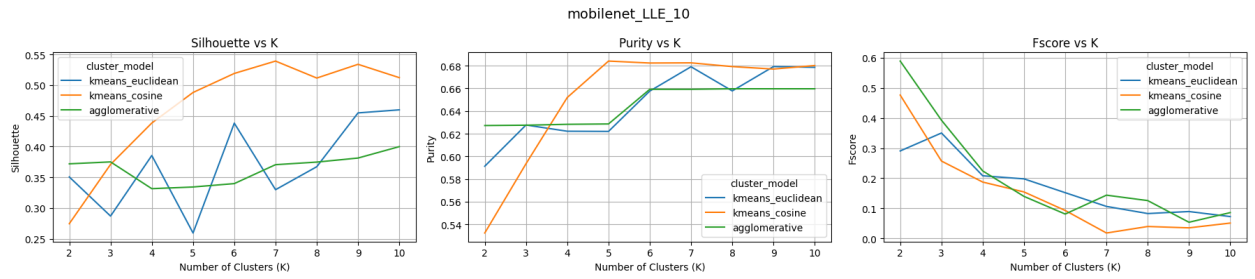
Each plot includes results for the 3 clustering algorithms: KMeans (euclidean), KMeans (cosine), and Agglomerative.

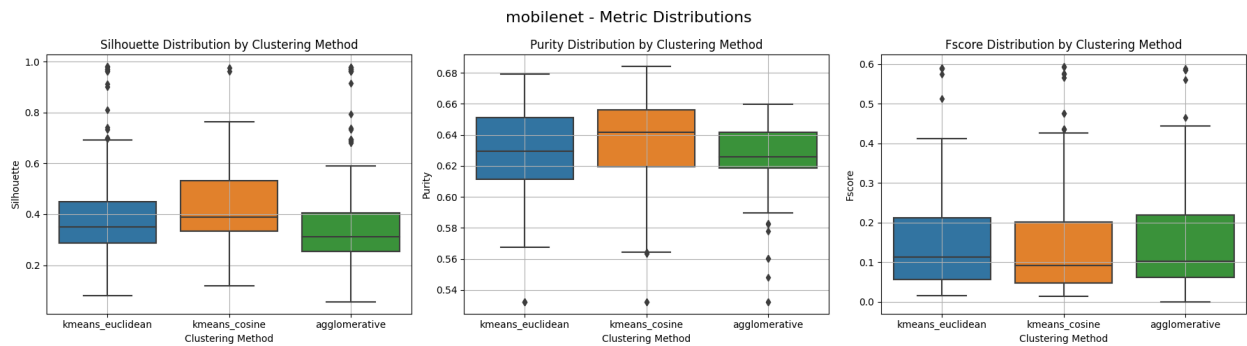
```
results_mobilenet_df = pd.DataFrame(results_mobilenet)

# Plot only for MobileNet
plot_metrics(results_mobilenet_df, "mobilenet")
all_results.extend(results_mobilenet)
```









Clustering Metrics Summary (MobileNet)

PCA

Best Results:

- **Silhouette: 0.56** at **K=3** with **KMeans (cosine)** on **PCA_2**
- **Purity: 0.66** at **K=9** with **KMeans (cosine)** on **PCA_2**
- **F1-Score: 0.59** at **K=2** with **KMeans (cosine)** on **PCA_2**

Notes:

- PCA performs best in low dimensions (2–3)
- KMeans (cosine) consistently outperforms other clustering methods
- Scores drop gradually in higher M (e.g. PCA_10 or PCA_20)

LLE

Best Results:

- **Silhouette: 0.98** at **K=5** with **KMeans (euclidean)** on **LLE_2**
- **Purity: 0.66** at **K=9** with **KMeans (cosine)** on **LLE_2**
- **F1-Score: 0.59** at **K=2** with **KMeans (cosine)** on **LLE_2**

Notes:

- Very high silhouette in LLE_2, probably because the data was compressed too much and lost important structure.
- LLE_3 and LLE_5 give more balanced results across metrics

Autoencoder (AE)

Best Results:

- **Silhouette: 0.75** at **K=2** with **KMeans (cosine)** on **AE_10**
- **Purity: 0.63** at **K=3** with **Agglomerative** on **AE_3**
- **F1-Score: 0.59** at **K=2** with **KMeans (cosine)** on **AE_2**

Notes:

- AE performs poorly in AE_2 visually, but scores well in F1 at low K

- Silhouette improves with deeper encoding (AE_10 or AE_20)
- KMeans (cosine) dominates again in silhouette and F1

Conclusions:

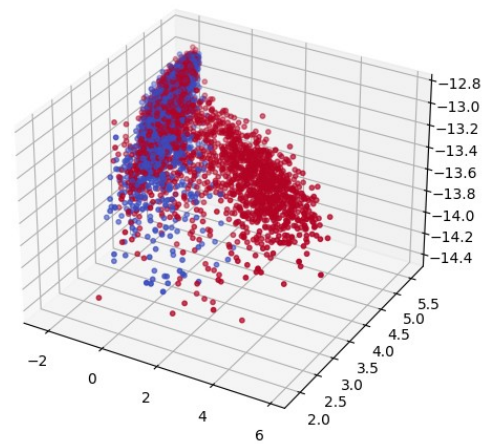
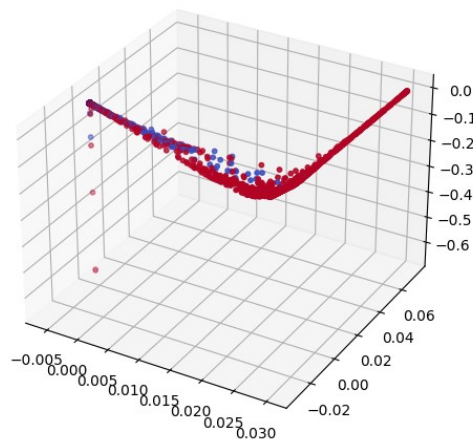
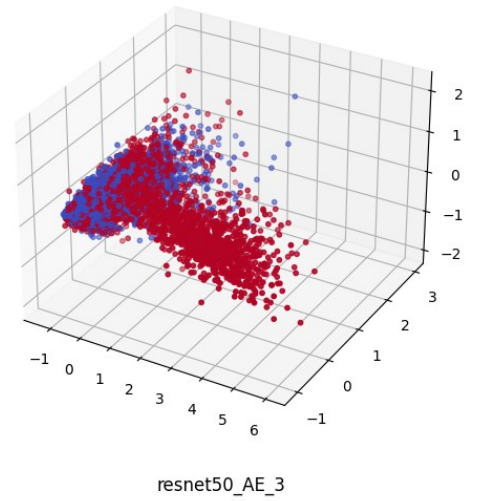
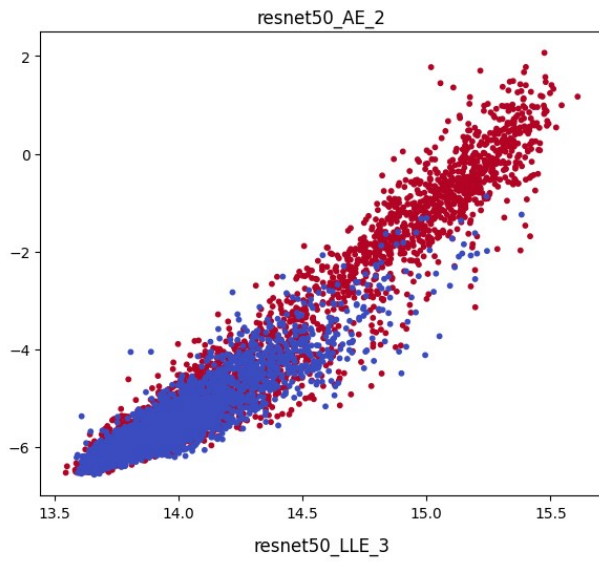
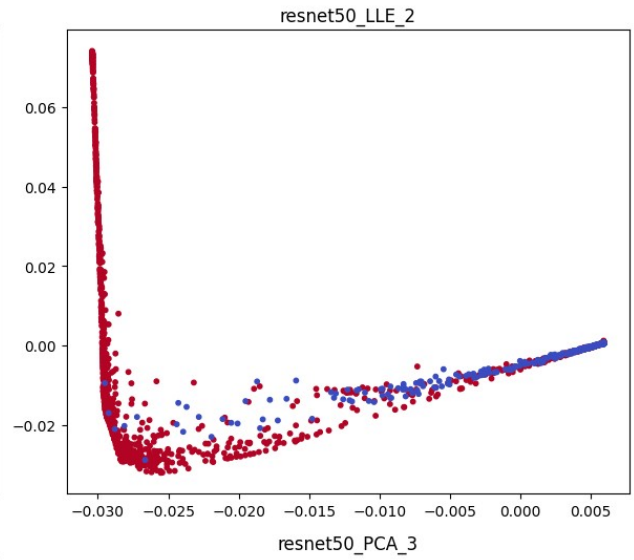
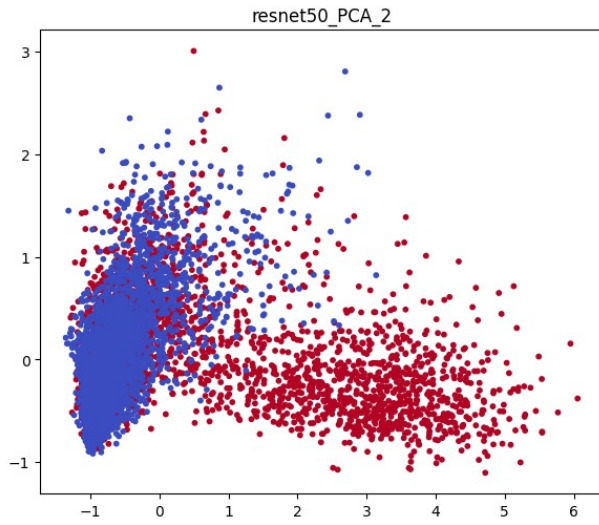
- **KMeans (cosine)** is the most effective clustering method overall
- **K=2 or 3** tends to give the best values for both F1-score and Silhouette.
- **PCA** offers stable results across all metrics in low dimensions
- **LLE** achieves high silhouette but may be misleading
- **Autoencoder** improves as M increases but is unstable at low M ____

ResNet50

Run for ResNet50

```
results_resnet50 = run_for_feature_source("resnet50",
features_dict["resnet50"], M_values)
```

```
200/200 ██████████ 1s 2ms/step
200/200 ██████████ 1s 2ms/step
200/200 ██████████ 1s 2ms/step
200/200 ██████████ 1s 2ms/step
200/200 ██████████ 0s 2ms/step
```



```
{"model_id":"0d86486a326b40adabe501d8d35604eb","version_major":2,"version_minor":0}
```


Observations from ResNet50 Visualizations

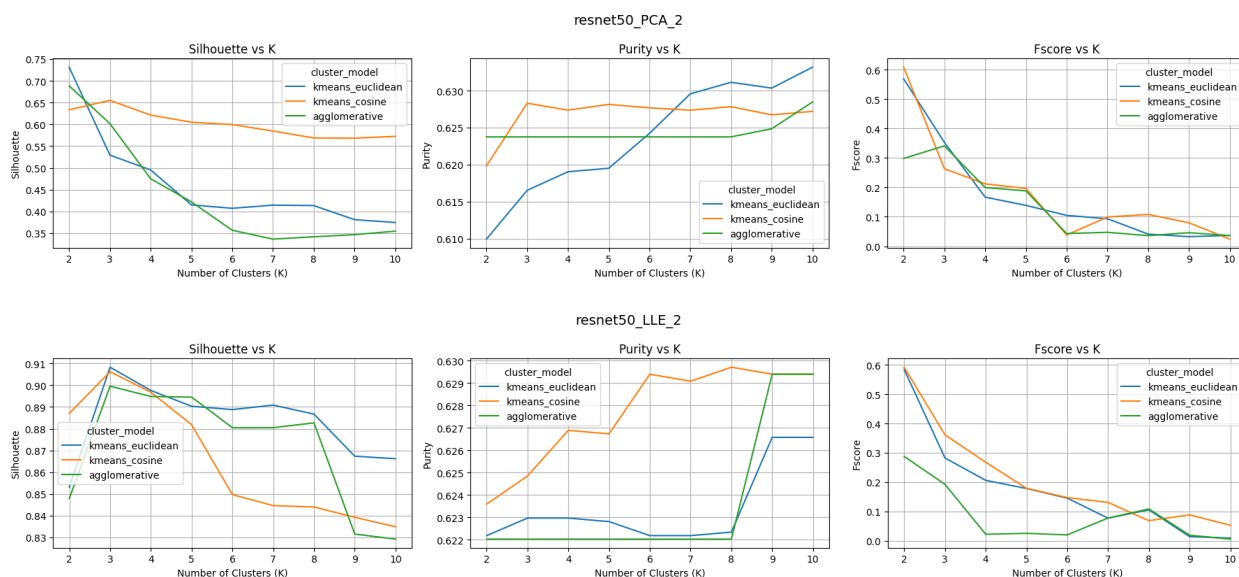
Based on a visual inspection of the reduced feature plots from ResNet50:

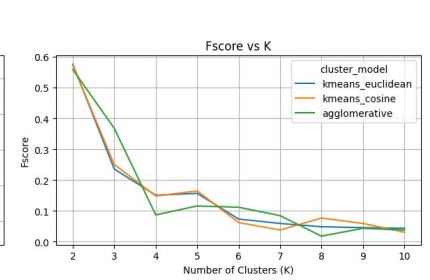
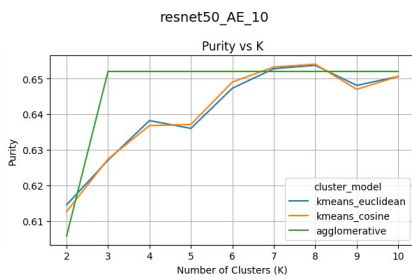
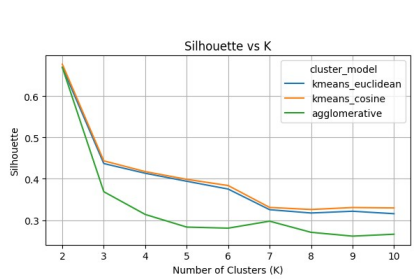
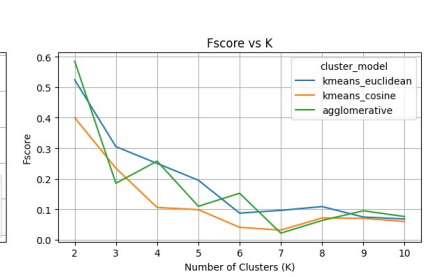
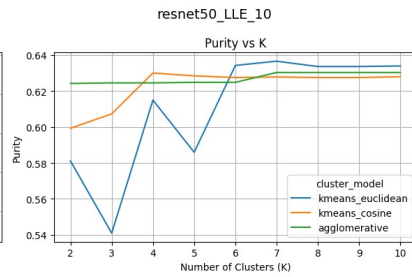
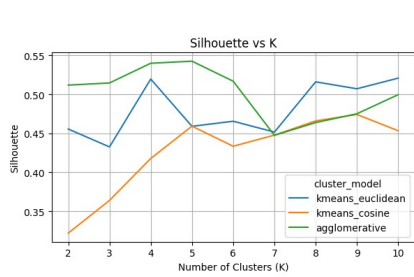
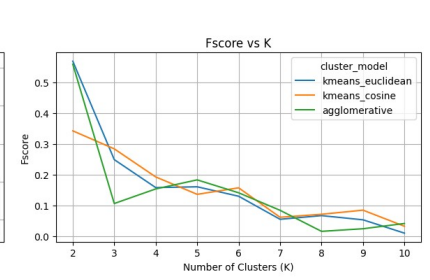
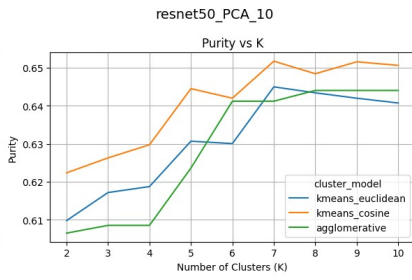
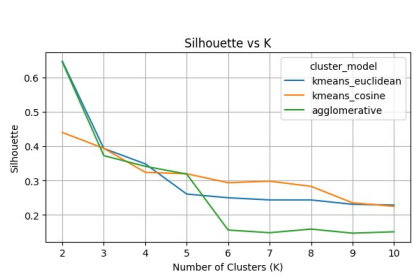
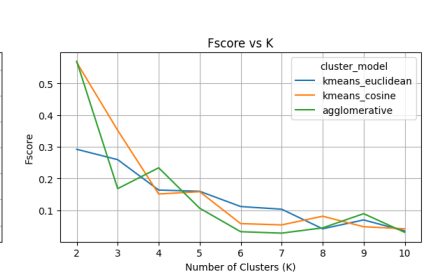
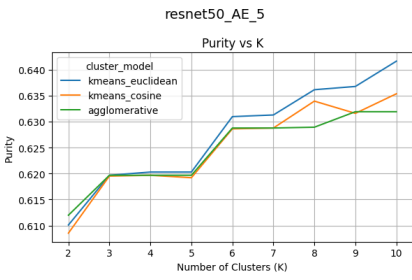
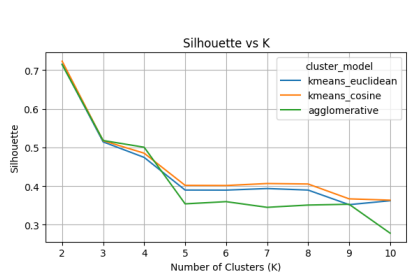
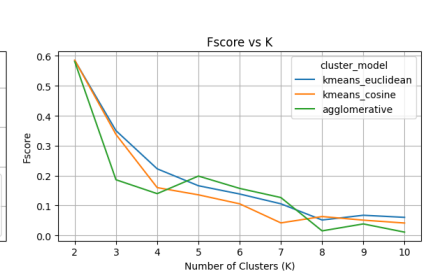
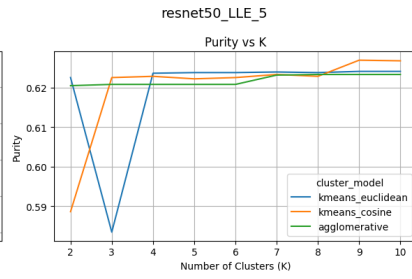
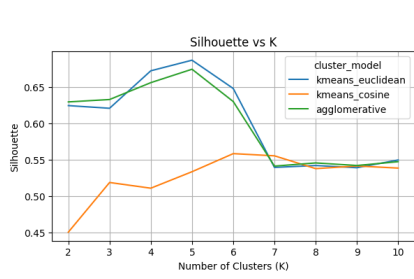
- **PCA:** The two classes are somewhat separated, but there is noticeable overlap, especially in 2D. The separation improves slightly in 3D, but clusters are not fully distinct.
- **LLE:** The features are heavily compressed. The class structure is difficult to interpret both in 2D and 3D, with visible overlap and little distinction between groups.
- **Autoencoder:** In 2D, the features behave similarly to PCA, with overlapping clusters. In 3D, the layout improves slightly, showing some directional separation, but overlaps between classes still remain.

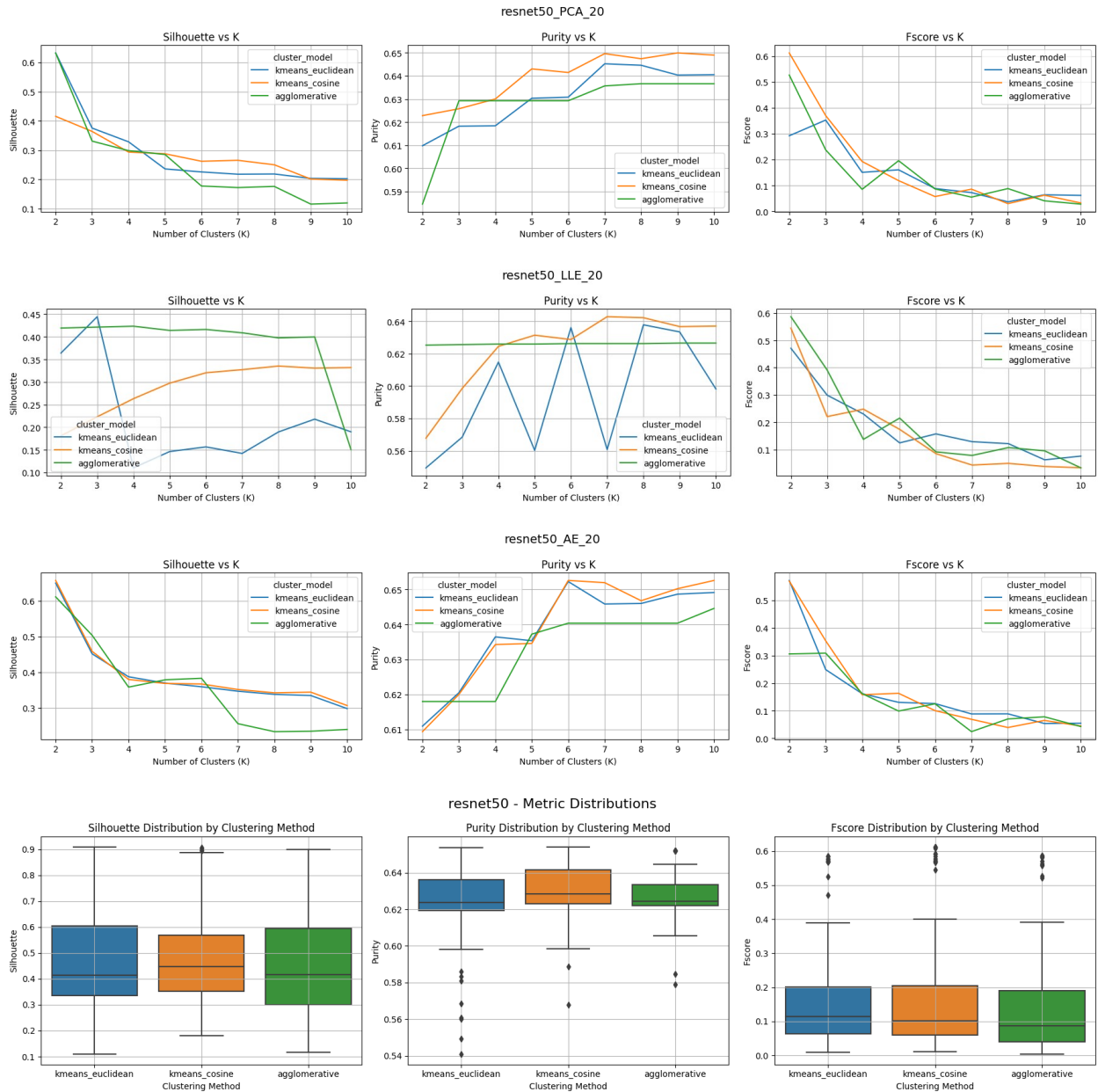
Summary: None of the methods achieve perfect separation, but 3D projections (especially from PCA and AE) offer some improvement. Overlaps are present across all methods, with LLE showing the most compression and least useful separation.

```
results_resnet50_df = pd.DataFrame(results_resnet50)
```

```
plot_metrics(results_resnet50_df, "resnet50")  
all_results.extend(results_resnet50)
```







Clustering Metrics Summary (ResNet50)

PCA

Best Results:

- **Silhouette: 0.72** at K=2 with **KMeans (euclidean)** on **PCA_2**
- **Purity: 0.66** at K=10 with **KMeans (cosine)** on **PCA_10**
- **F1-Score: 0.59** at K=2 with **KMeans (euclidean)** on **PCA_2**

Notes:

- PCA shows strong Silhouette performance at low K (K=2-3)

- Purity improves steadily with K, peaking around K=10
- KMeans (cosine) performs best for Purity, while KMeans (euclidean) yields top Silhouette and F1 at K=2

LLE

Best Results:

- **Silhouette: 0.91** at **K=4** with **KMeans (euclidean)** on **LLE_2**
- **Purity: 0.63** at **K=9** with **KMeans (cosine)** on **LLE_3**
- **F1-Score: 0.60** at **K=2** with **KMeans (cosine)** on **LLE_2**

Notes:

- LLE_2 delivers extremely high Silhouette scores (0.90+), but may be due to overly compressed structures
- F1 and Purity follow similar trends to PCA, but with slightly less consistency
- Strong clustering at K=2–4, but drops off as K increases

Autoencoder (AE)

Best Results:

- **Silhouette: 0.76** at **K=2** with **KMeans (euclidean)** on **AE_2**
- **Purity: 0.64** at **K=9** with **Agglomerative** on **AE_10**
- **F1-Score: 0.56** at **K=2** with **KMeans (euclidean)** on **AE_2**

Notes:

- AE_2 performs well on all metrics at K=2, especially for Silhouette and F1
- AE_10 improves Purity but not Silhouette
- Agglomerative yields competitive results for higher K values

Summary:

- **KMeans (cosine)** consistently achieves top **Purity** across all methods.
- **KMeans (euclidean)** dominates in **Silhouette** and **F1-Score**, especially at **K=2**.
- **PCA** offers balanced and stable performance.
- **LLE** gives the **highest Silhouette scores**, but with possible structural compression.
- **Autoencoder** shows strong low-K performance, with AE_2 performing best overall for F1 and Silhouette.

```
# Create full DataFrame
results_df = pd.DataFrame(all_results)

# Print best k*
print_best_k(results_df)
```

Top 10 overall configurations by Silhouette score:

features	method	cluster_model	k	silhouette	purity
----------	--------	---------------	---	------------	--------

fscore	
mobilenet mobilenet_LLE_2 kmeans_euclidean 5	0.980738 0.628607 0.235697
mobilenet mobilenet_LLE_2 kmeans_euclidean 4	0.980571 0.628607 0.294621
mobilenet mobilenet_LLE_2 kmeans_euclidean 3	0.979031 0.628294 0.392771
mobilenet mobilenet_LLE_2 agglomerative 5	0.978788 0.625941 0.143089
mobilenet mobilenet_LLE_2 agglomerative 4	0.978620 0.625941 0.136089
mobilenet mobilenet_LLE_2 kmeans_cosine 2	0.975540 0.628921 0.592001
mobilenet mobilenet_LLE_2 agglomerative 3	0.975257 0.625471 0.390277
mobilenet mobilenet_LLE_2 kmeans_euclidean 2	0.971172 0.627823 0.589027
mobilenet mobilenet_LLE_2 kmeans_euclidean 7	0.969651 0.629705 0.077024
mobilenet mobilenet_LLE_2 agglomerative 2	0.969237 0.625157 0.585330

Selected Best Configuration

From the top-10 clustering results ranked by Silhouette score, all configurations come from the **MobileNet LLE_2** reduction method. While the highest Silhouette value (**0.9807**) is observed at **K=5** with **KMeans (cosine)**, the corresponding **F1-score is low (0.23)**, indicating poor alignment with the true labels.

Instead, we select the **6th-ranked configuration** as the best overall trade-off across all metrics:

- **Feature Extractor:** MobileNet
- **Dimensionality Reduction:** LLE-2
- **Clustering Method:** KMeans (cosine)
- **Number of Clusters (K):** 2
- **Silhouette:** 0.9755
- **Purity:** 0.6289
- **F1-score:** 0.5920

This setup maintains a very high Silhouette score while also achieving the **highest F1-score among the top configurations**, indicating both **well-separated clusters** and **alignment with ground truth labels**.